

Complete

PAST YEA

CLL Q.P M

Shiddat T

Debuggin

Search Ti

Cream an

sequence

AC CIRCU

+

Ask Gemini

chatgpt.com/c/69941ec2-5890-8324-a575-4eb9039a9a2e

☆

9

Relaunch to update

ChatGPT

Get Plus

Share

I need to create a movie ticket system in C++ with operations like BOOK CANCEL IS_BOOKED and AVAILABLE_TICKETS each movie has 100 tickets How should I solve this using classes what classes should be used to store bookings more easily and add functions to check booking and count available tickets

You can write it like this — simple and clear

To solve this, I will create a class **MovieTicket** to handle all operations.

Inside the class, I need to store which user booked which movie. For this, I can use a **map of set**:

- `map<int, set<int>>`
 - key = movie ID
 - value = set of users who booked that movie

Using a set helps because it automatically avoids duplicate bookings.

Functions I will create:

1. BOOK(x, y)

- Check if user `x` already booked movie `y` → if yes, return false
- Check if total bookings for movie `y` reaches 100 → if yes, return false
- Otherwise, add user to the set and return true

2. CANCEL(x, y)

Ask anything

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).

day1.cpp

day2.cpp

day3.cpp

day4.cpp

day5.cpp

day6.cpp

day8.cpp

movieTicket.cpp

day7.cpp

▶ 📄 ...

G- movieTicket.cpp

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class MovieTicket {
5  private:
6      // Using unordered_map for O(1) average access
7      // Using unordered_set for O(1) average insert/delete/lookup
8      unordered_map<int, unordered_set<int>> bookings;
9      const int MAX_SLOTS = 100;
10
11 public:
12     // — BOOK X Y —
13     // Try to book a ticket for customer X for movie Y
14     // Returns false if:
15     // 1. Customer already booked this movie
16     // 2. Movie is sold out (100 tickets gone)
17     // Otherwise adds customer to the movie's booking set → true
18     bool book(int customerId, int movieId) {
19         // If movieId not seen before, this auto-creates an empty set for it
20         unordered_set<int>& bookedSet = bookings[movieId];
21
22         // Condition 1: duplicate booking
23         if (bookedSet.count(customerId)) return false;
24
25         // Condition 2: sold out
26         if ((int)bookedSet.size() >= MAX_SLOTS) return false;
27
28         // All clear – book the ticket
29         bookedSet.insert(customerId);
30         return true;
31     }
32
33     // — CANCEL X Y —
34     // Try to cancel customer X's ticket for movie Y
35     // Returns false if customer never booked this movie
36     // Otherwise removes them from the set → true
37     bool cancel(int customerId, int movieId) {
38         // If movieId has no entry at all, customer definitely hasn't booked
39         if (bookings.find(movieId) == bookings.end()) return false;
40
41         unordered_set<int>& bookedSet = bookings[movieId];
42
43         // erase() returns number of elements removed (0 or 1 for a set)
44         // So it naturally returns true/false for us
```

```

65 int main() {
66     ios_base::sync_with_stdio(false);
67     cin.tie(NULL);
68
69     int Q;
70     cin >> Q;
71     cin.ignore(); // consume the newline after the integer
72
73     MovieTicket mt;
74
75     while (Q--) {
76         string line;
77         getline(cin, line);
78         istringstream iss(line);
79
80         string command;
81         iss >> command;
82
83         if (command == "BOOK") {

```

```

65 int main() {
66     ios_base::sync_with_stdio(false);
67     cin.tie(NULL);
68
69     int Q;
70     cin >> Q;
71     cin.ignore(); // consume the newline after the integer
72
73     MovieTicket mt;
74
75     while (Q--) {
76         string line;
77         getline(cin, line);
78         istringstream iss(line);
79
80         string command;
81         iss >> command;
82
83         if (command == "BOOK") {
84             int x, y;
85             iss >> x >> y;
86             cout << (mt.book(x, y) ? "true" : "false") << "\n";
87
88         } else if (command == "CANCEL") {
89             int x, y;
90             iss >> x >> y;
91             cout << (mt.cancel(x, y) ? "true" : "false") << "\n";
92
93         } else if (command == "IS_BOOKED") {
94             int x, y;
95             iss >> x >> y;
96             cout << (mt.isBooked(x, y) ? "true" : "false") << "\n";
97
98         } else if (command == "AVAILABLE_TICKETS") {
99             int y;
100             iss >> y;
101             cout << mt.availableTickets(y) << "\n";
102         }
103     }
104
105     return 0;
106 }

```

← →

🔍 C++ class

🔄

📄

🔍

🔗

🔧

📁

👤

⚙️

day1.cpp day2.cpp day3.cpp day4.cpp day5.cpp day6.cpp day8.cpp movieTicket.cpp × day7.cpp

movieTicket.cpp

15 public:
22 bool book(int customerId, int movieId) {
29 // Condition 2: sold out
30 if ((int)bookedSet.size() >= MAX_SLOTS) return false;
31
32 // All clear - book the ticket
33 bookedSet.insert(customerId);
34 return true;
35 }
36
37 // — CANCEL X Y —
38 // Try to cancel customer X's ticket for movie Y
39 // Returns false if customer never booked this movie
40 // Otherwise removes them from the set → true
41 bool cancel(int customerId, int movieId) {

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

zsh + ▾ 🗑️ ⋮ | 📄 ×

● gaurisharma@MacBookAir C++ class % g++ movieTicket.cpp
● gaurisharma@MacBookAir C++ class % ./a.out
6
BOOK 1 10
true
BOOK 2 10
true
IS_BOOKED 1 10
true
BOOK 1 10
false
CANCEL 1 10
true
AVAILABLE_TICKETS 10
99
○ gaurisharma@MacBookAir C++ class %

Ln 14, Col 1 Spaces: 4 UTF-8 LF {} C++ 🏠 Finish Setup 🔔

```
1 // Ticket Booking System
2 #include <iostream>
3 #include <set>
4 #include <string>
5 using namespace std;
6
7 const int MAX_SLOTS = 10;
8
9 class TicketBookingSystem {
10 public:
11     set<int> bookedSet;
12     int availableTickets = MAX_SLOTS;
13
14     bool book(int customerId, int movieId) {
15         // Condition 1: Check if movie is already booked
16         if (bookedSet.find(movieId) != bookedSet.end()) {
17             return false;
18         }
19         // Condition 2: sold out
20         if ((int)bookedSet.size() >= MAX_SLOTS) return false;
21
22         // All clear - book the ticket
23         bookedSet.insert(customerId);
24         return true;
25     }
26
27     bool cancel(int customerId, int movieId) {
28         // Try to cancel customer X's ticket for movie Y
29         // Returns false if customer never booked this movie
30         // Otherwise removes them from the set → true
31         if (bookedSet.find(customerId) == bookedSet.end()) {
32             return false;
33         }
34         bookedSet.erase(customerId);
35         return true;
36     }
37
38     int getAvailableTickets() {
39         return availableTickets;
40     }
41
42     void displayStatus() {
43         cout << "Available Tickets: " << availableTickets << endl;
44         cout << "Booked Tickets: ";
45         for (int id : bookedSet) {
46             cout << id << " ";
47         }
48         cout << endl;
49     }
50 };
51
52 int main() {
53     TicketBookingSystem tbs;
54
55     // Test Case 1: Successful booking
56     tbs.book(1, 10);
57     tbs.book(2, 10);
58     tbs.displayStatus();
59
60     // Test Case 2: Cancel booking
61     tbs.cancel(1, 10);
62     tbs.displayStatus();
63
64     // Test Case 3: Check available tickets
65     int available = tbs.getAvailableTickets();
66     cout << "Available Tickets: " << available << endl;
67
68     return 0;
69 }
```